

药物预警微信小程序

技术文档

微信云开发 · 药品效期管理 · 多模态 AI 辅助录入

文档说明：本文档完全依据当前的代码、配置和数据库设计说明编写。文中会明确标注已经实现、依赖云端配置以及尚未完整落地的能力。

目录

1. 项目概述	4
1.1 项目目标	4
1.2 功能边界	4
2. 技术栈与运行环境	5
2.1 关键配置	5
3. 总体架构设计	5
3.1 分层职责	6
3.2 设计特点	6
4. 项目目录结构	6
5. 小程序端设计	7
5.1 全局配置与导航	7
5.2 登录页	8
5.3 首页	8
5.4 药品列表	9
5.5 手动新增与编辑	10
5.6 拍照录入	11
5.7 语音录入	11
5.8 AI 对话	13
5.9 消息中心	13
5.10 个人中心与设置	14
6. 公共服务层	14
6.1 字段映射	16
7. 云函数设计	16
7.1 database 云函数	16
7.2 参数规范化	17

7.3 aiAssistant 云函数	18
7.4 quickstartFunctions	19
8. 数据库与云存储设计	19
8.1 drugs 集合	19
8.2 users 集合	20
8.3 推荐索引	20
8.4 云存储路径	21
9. 核心业务流程	21
9.1 手动新增	21
9.2 拍照识别	21
9.3 语音识别	21
9.4 提醒生成	22
9.5 数据刷新	22
10. 效期计算与状态规则	22
11. 本地存储与状态管理	23
12. AI 能力设计	23
12.1 输出约束	23
12.2 模型回退	24
12.3 安全提示	24
12.4 配置管理	24
13. 异常处理与降级	24
14. 未来优化方向	24
14.1 安全与正确性	24
14.2 提醒与同步	25
14.3 工程化	25
14.4 产品能力	25

1. 项目概述

“药物预警”是一个家庭药箱效期管理微信小程序。系统以药品库存记录为核心，提供手动录入、拍照识别、语音识别、药品列表、过期/临期提醒、AI 问答、用户资料和偏好设置等功能。数据存储在微信云开发数据库中，图片、录音和头像存储在云存储中，后端业务通过云函数统一处理。

1.1 项目目标

- 建立按微信用户隔离的家庭药品云端清单。
- 以到期日期为基础自动计算剩余天数，并将药品划分为正常、临期、过期和日期未知四种状态。
- 降低药品信息录入成本，支持手工、图像、语音和自然语言四种入口。
- 将过期和三天内临期药品集中展示，帮助用户优先处理高风险库存。
- 通过 AI 根据当前药箱数据提供清单汇总、保存建议和一般性管理建议。

1.2 功能边界

重要边界：系统定位是药箱库存与效期管理工具，不是医疗诊断、处方审核或用药决策系统。AI 云函数会附加安全提示，但业务上仍应要求用户核对识别结果，并在专业用药问题上咨询医生或药师。

类别	当前实现
账号	微信登录换取会话 <code>code</code> ；实际用户身份由云函数中的 <code>OPENID</code> 确定；本地保存 7 天登录标记
药品管理	新增、批量新增、查询、编辑、软删除、搜索、筛选和排序
AI 录入	图片识别、录音转写、文本结构化解析，识别结果必须经过前端确认
提醒	首页、药品列表和消息中心展示过期/临期信息；当前不是系统级定时推送
资料与设置	用户昵称、头像、简介云端保存；提醒偏好和同步开关主要保存在本地

2. 技术栈与运行环境

层级	技术/组件	用途
客户端	微信小程序原生框架	WXML 页面结构、WXSS 样式、JavaScript 页面逻辑、JSON 配置
云平台	微信云开发	云数据库、云函数、云存储、微信身份上下文
服务端运行时	Node.js + wx-server-sdk ~3.0.4	database 与 aiAssistant 云函数
文本模型	千问 qwen-plus, 智谱 GLM 作为回退	AI 对话、药品文本结构化解析
视觉模型	qwen-vl-plus	药品包装图片识别
语音识别	DashScope paraformer-v2	录音文件异步转写
开发工具	微信开发者工具, 基础库 3.16.1	编译、预览、云函数部署和调试

2.1 关键配置

- 小程序 AppID: 配置文件中为 wxdbc374430f3cdb15。
- 云环境 ID: app.js 中固定为 cloud1-d6g8tzu652c61fd0f。
- 小程序根目录: uploadCloudFunction/miniprogram/。
- 云函数根目录: uploadCloudFunction/cloudfunctions/。
- app.js 要求微信基础库不低于 2.2.3; 当前项目配置基础库为 3.16.1。

3. 总体架构设计

系统采用典型的微信云开发三层结构: 小程序页面负责交互和展示, 公共服务层负责数据映射, 云函数负责身份校验、数据规范化和外部 AI API 调用, 云数据库与云存储负责持久化。

```
wx.cloud.init({
  env: "cloud1-d6g8tzu652c61fd0f",
  traceUser: true
});
```

3.1 分层职责

层级	职责	主要文件
表示层	页面展示、输入校验、加载状态、Toast/Modal、页面跳转	miniprogram/pages/**
客户端服务层	封装 database 云函数调用，完成前后端字段映射和效期辅助计算	miniprogram/utils/drugService.js
业务云函数层	用户隔离、参数校验、数据规范化、CRUD、文件权限检查	cloudfunctions/database/index.js
AI 集成层	对话、图像识别、语音转写、文本解析、模型回退与结果归一化	cloudfunctions/aiAssistant/index.js
数据层	药品、用户资料、媒体文件	drugs、users、云存储路径

3.2 设计特点

- 数据库写操作集中在云函数，避免前端直接写库导致的字段漂移。
- 所有药品查询和更新都附带 `_openid` 条件，实现用户级数据隔离。
- 以 `expireAtTs` 作为稳定排序字段，以 `daysToExpire` 作为可刷新展示快照。
- AI 输出仅作为草稿，前端提供低置信度、缺字段和异常日期警告。
- 删除采用软删除，保留记录审计信息；附件删除为可选操作。

4. 项目目录结构

```

finally/
|-- project.config.json          # 工作区入口配置
|-- logo.jpg
|-- uploadCloudFunction/
|   |-- miniprogram/
|   |   |-- app.js / app.json / app.wxss
|   |   |-- pages/          # 13 个页面
|   |   |-- utils/drugService.js    # 药品服务适配层
|   |   |-- components/cloudTipModal/ # 云开发提示组件
|   |   |-- images/          # 位图与 SVG 图标
|   |-- cloudfunctions/
|   |   |-- database/        # 药品、用户和文件管理
|   |   |-- aiAssistant/     # AI 对话、视觉、语音、文本解析
|   |   |-- quickstartFunctions/  # 云开发示例函数，非核心业务
|-- docs/database-design.md     # 数据库设计说明
|-- project.config.json

```

实际业务入口应使用根目录 `project.config.json`，该配置已将 `miniprogramRoot` 和 `cloudfunctionRoot` 指向 `uploadCloudFunction` 下对应目录。`uploadCloudFunction` 内还保留一份独立项目配置，可用于直接打开子项目。

5. 小程序端设计

5.1 全局配置与导航

`app.js` 在启动时检查 `wx.cloud` 能力并初始化指定云环境，开启 `traceUser`。`app.json` 注册 13 个页面，并将首页、药品、AI、我的设置为四个 `TabBar` 页面。非 `TabBar` 页面通过 `navigateTo` 进入。

```
"pages": [
  "pages/login/login",
  "pages/home/home",
  "pages/drug-list/drug-list",
  "pages/ai-chat/ai-chat",
  "pages/profile/profile",
  "pages/edit-drug/edit-drug",
  "pages/manual-input/manual-input",
  "pages/capture-input/capture-input",
  "pages/voice-input/voice-input",
  "pages/settings/settings",
  "pages/message-center/message-center",
  "pages/about/about",
  "pages/privacy/privacy"
],

"tabBar": {
  "color": "#7B8496",
  "selectedColor": "#4F7CFF",
  "backgroundColor": "#FFFFFF",
  "borderStyle": "black",
  "list": [
    {
      "pagePath": "pages/home/home",
      "text": "首页",
      "iconPath": "images/icons/home.png",
      "selectedIconPath": "images/icons/home-active.png"
    },
    {
      "pagePath": "pages/drug-list/drug-list",
      "text": "药物",
      "iconPath": "images/icons/goods.png",
      "selectedIconPath": "images/icons/goods-active.png"
    }
  ]
}
```

路由	页面职责	主要依赖
<code>pages/login/login</code>	登录、隐私授权、用户协议入口、7 天本地登录判断	<code>database.getUserProfile</code>
<code>pages/home/home</code>	药箱概览、状态统计、前三条紧急提醒、三种新增入口	<code>drugService.listDrugs</code>
<code>pages/drug-list/drug-list</code>	列表、搜索、筛选、排序、编辑、删除和聚焦记录	<code>drugService CRUD</code>
<code>pages/ai-chat/ai-chat</code>	AI 对话、快捷问题、个性化风险提示、聊天缓存	<code>aiAssistant.chat</code>
<code>pages/profile/profile</code>	用户资料、统计、健康评分、	<code>database 用户接口</code>

路由	页面职责	主要依赖
	数据同步、设置与支持入口	
pages/manual-input/manual-input	手动录入药名、数量、剂型、分类、开封状态和日期	drugService.createDrug
pages/edit-drug/edit-drug	读取并更新单条药品	getDrug/updateDrug
pages/capture-input/capture-input	相机/相册选图、云上传、视觉识别、多条确认和批量保存	visionRecognize
pages/voice-input/voice-input	录音、语音转写、文本解析、单条确认保存	voiceRecognize/parseDrugText
pages/message-center/message-center	根据药品数据实时生成站内提醒，维护已读和删除状态	本地 messageState
pages/settings/settings	提醒开关、同步开关、提醒时间、本地缓存清理	本地 settings
pages/about/about	版本、联系方式、说明信息	小程序系统信息
pages/privacy/privacy	隐私政策展示	静态内容

5.2 登录页

- 页面加载时读取 isLogin 和 loginTime；若登录标记为字符串 true 且未超过 7 天，直接切换到首页。
- 用户点击微信登录后先展示隐私确认弹窗，同意后调用 wx.login。
- wx.login 返回 code 仅用于确认微信登录调用成功；云函数身份实际来自 cloud.getWXContext().OPENID。
- 调用 getUserProfile 获取或创建默认资料，并写入 userInfo、isLogin、loginTime。
- 退出登录只清除本地登录信息，不删除云端用户与药品数据。

5.3 首页

首页每次 onLoad 和 onShow 都拉取药品列表，统计 expired、expiring、normal 数量。提醒列表仅保留过期和临期药品，按紧急程度排序后截取前三条。用户点击状态卡片时，通过 drugFilter 临时存储向药品列表传递筛选条件。

```

async loadDrugsData() {
  try {
    const drugs = await drugService.listDrugs();
    const expired = drugs.filter(item => item.status === "expired").length;
    const expiring = drugs.filter(item => item.status === "expiring").length;
    const normal = drugs.filter(item => item.status === "normal").length;
    const focusCount = expired + expiring;
    const reminderList = this.getReminderList(drugs);

    this.setData({
      expiredCount: expired,
      expiringCount: expiring,
      normalCount: normal,
      wasteText: `当前需关注 ${focusCount} 件药品`,
      priorityText: expiring > 0
        ? `${expiring} 件药品即将到期，建议优先处理。`
        : "暂无紧急临期药品，继续保持定期整理。",
      isReminderEmpty: reminderList.length === 0,
      reminderList
    });
  }

  getReminderList(drugs) {
    return drugs
      .filter(item => item.status === "expired" || item.status === "expiring")
      .sort((a, b) => this.getUrgencyValue(a) - this.getUrgencyValue(b))
      .slice(0, 3)
      .map(item => ({
        id: item.id,
        name: item.name || "未命名药品",
        openState: item.openState || "未开封",
        tagText: this.getReminderTag(item),
        tagClass: item.status === "expired" ? "tag-danger" : "tag-warning"
      }));
  }
}

```

5.4 药品列表

```

filterDrugs(drugs) {
  switch (this.data.currentFilter) {
    case "attention":
      return drugs.filter(drug => drug.status === "expired" || drug.status === "expiring");
    case "expired":
      return drugs.filter(drug => drug.status === "expired");
    case "expiring":
      return drugs.filter(drug => drug.status === "expiring");
    case "sealed":
      return drugs.filter(drug => drug.openState === "未开封");
    case "opened":
      return drugs.filter(drug => drug.openState === "已开封");
    default:
      return drugs;
  }
}

```

```
sortDrugs(drugs) {
  const sorted = [...drugs];
  if (this.data.currentSort === "name") {
    return sorted.sort((a, b) => String(a.name || "").localeCompare(String(b.name || ""),
      "zh-CN"));
  }
  if (this.data.currentSort === "createdAt") {
    return sorted.sort((a, b) => Number(b.createdAt || 0) - Number(a.createdAt || 0));
  }
  return sorted.sort((a, b) => this.getSortDays(a) - this.getSortDays(b));
}
```

功能	规则
筛选	全部、需处理、已过期、临期、未开封、已开封。
搜索	在前端对药名、数量、开封状态、状态标签和剂型文本做包含匹配。
排序	到期倒计时、名称中文排序、创建时间倒序。
聚焦	从首页或消息中心写入 focusDrugId，列表读取后为目标项增加 focus 样式。
删除	弹窗确认后调用 deleteDrug，服务端软删除，前端同步移除本地列表项。

5.5 手动新增与编辑

```
async saveDrug() {
  if (this.data.isSaving) return;

  const drug = this.data.drug;
  if (!drug.name.trim()) return wx.showToast({ title: "请填写药品名称", icon: "none" });
  if (!drug.qty.trim()) return wx.showToast({ title: "请填写数量", icon: "none" });
  if (!drug.date) return wx.showToast({ title: "请选择到期日期", icon: "none" });

  const payload = {
    ...drug,
    name: drug.name.trim(),
    qty: drug.qty.trim(),
    openState: this.data.showOpenState ? drug.openState : "独立包装",
    source: "manual"
  };

  this.setData({ isSaving: true });
  wx.showLoading({ title: "保存中...", mask: true });
```

手动新增和编辑页面采用相似表单。剂型为瓶装或软膏时显示开封状态；板装和袋装等独立包装会强制写入“独立包装”。保存前要求药名、数量和到期日期均非空。日期变化后立即调用 `drugService.getDaysRemaining` 预览状态。

5.6 拍照录入

```
takePhoto() {
  if (this.data.isRecognizing) return;

  if (!this.cameraContext) {
    this.cameraContext = wx.createCameraContext();
  }

  this.cameraContext.takePhoto({
    quality: "high",
    success: res => {
      this.recognizeImage(res.tempImagePath);
    },
    fail: err => {
      console.error("拍照失败", err);
      wx.showToast({ title: "无法打开摄像头，请检查授权", icon: "none" });
    }
  });
}

saveAllDrugs() {
  const items = this.data.detectedItems.map(item => this.withAiChecks(item));

  for (let i = 0; i < items.length; i++) {
    if (!items[i].name.trim()) return wx.showToast({ title: `第 ${i + 1} 项请填写药名`, icon: "none" });
    if (!items[i].qty.trim()) return wx.showToast({ title: `第 ${i + 1} 项请填写数量`, icon: "none" });
    if (!items[i].date) return wx.showToast({ title: `第 ${i + 1} 项请选择日期`, icon: "none" });
  }
}
```

1. 调用相机或相册选择图片。
2. 上传到 ai-inputs/images/{timestamp}-{random}.{ext}。
3. 调用 aiAssistant.visionRecognize，并接收最多 8 条归一化药品候选。
4. 为每条候选显示置信度，低于 0.45 标记为低，0.45 到 0.75 标记为中，其余为高。
5. 检查药名、数量、日期和日期合理性；用户可以修改、增加或删除候选项。
6. 确认后调用 batchCreateDrugs 批量保存，图片 fileID 作为每条记录的附件。

5.7 语音录入

```
uploadAudio(filePath) {
  ...
  return wx.cloud.uploadFile({
    cloudPath: `ai-inputs/audio/${Date.now()}-${Math.random().toString(16).slice(2)}.mp3`,
    filePath
  }).then(res => res.fileID);
},
```

```

async recognizeVoice(filePath) {
  this.setData({
    isRecognizing: true,
    micDesc: "正在识别语音..."
  });
  wx.showLoading({ title: "识别中...", mask: true });
  try {
    const fileID = await this.uploadAudio(filePath);
    const res = await wx.cloud.callFunction({
      name: "aiAssistant",
      data: { action: "voiceRecognize", fileID }
    });
    const result = res.result || {};
    if (!result.success) throw new Error(result.message || "识别失败");
    const data = result.data || {};
    const item = Array.isArray(data.items) && data.items.length ? data.items[0] : {};
    this.setData({
      recognizedText: data.recognizedText || "",
      audioFileID: fileID
    });
  } catch (err) {
    console.error("语音识别失败", err);
  }
}

async parseTypedText() {
  const text = String(this.data.recognizedText || "").trim();
  if (!text) return wx.showToast({ title: "请先输入文字", icon: "none" });
  this.setData({
    isRecognizing: true,
    micDesc: "正在解析文字..."
  });
  wx.showLoading({ title: "解析中...", mask: true });
  try {
    const res = await wx.cloud.callFunction({
      name: "aiAssistant",
      data: { action: "parseDrugText", text }
    });
    const result = res.result || {};
    if (!result.success) throw new Error(result.message || "解析失败");
    const items = result.data && result.data.items;
    this.setDrug(this.normalizeDrug(items && items[0]));
    wx.showToast({ title: "解析成功", icon: "success" });
  } catch (err) {
    console.error("文字解析失败", err);
  }
}

```

语音页通过 `wx.getRecorderManager` 录制音频，上传到 `ai-inputs/audio/` 后调用 `voiceRecognize`。服务端先异步转写，再调用文本模型提取结构化药品字段。用户也可以直接输入自然语言并调用 `parseDrugText`。当前页面一次只采用识别结果中的第一条药品记录。

5.8 AI 对话

```

async sendPrompt(prompt, displayText) {
  if (this.data.isLoading) return;

  const userMessage = { role: "user", content: displayText };
  const messages = [...this.data.messages, userMessage];
  this.setData({ messages, inputText: "", isLoading: true, sendButtonClass: "disabled" });
  this.saveMessages(messages);
  this.scrollToBottom();

  try {
    const drugs = await drugService.listDrugs();
    const res = await wx.cloud.callFunction({
      name: "aiAssistant",

```

- 快捷问题包括快过期药品、今日处理清单和分类保存建议。
- 发送请求时附带当前药品列表和最近 8 条消息，使回答具备药箱上下文。
- 本地仅保留最近 30 条聊天消息，存储键为 aiChatMessages。
- 页面会基于过期/临期药品自动生成个性化提示；AI 服务失败时降级提示用户直接查看药品列表。

5.9 消息中心

```

buildExpiryMessages(drugs) {
  const state = this.getMessageState();
  const nowText = this.formatTime(new Date());

  return (drugs || []).
    .filter(item => this.shouldRemind(item))
    .sort((a, b) => this.getUrgencyValue(a) - this.getUrgencyValue(b))
    .map(item => this.toMessage(item, nowText, state))
    .filter(item => !state.deletedMap[item.id]);

  shouldRemind(item) {
    if (!item) return false;
    return item.status === "expired" || item.status === "expiring";
  }

```

```

getMessageContent(item, status, days) {
  const qtyText = item.qty ? `，库存 ${item.qty}` : "";
  const openText = item.openState ? `，状态 ${item.openState}` : "";
  if (status === "expired") {
    return `已过期 ${Math.abs(days)} 天${qtyText}${openText}，建议立即核对并停止继续使用。`;
  }
  return `剩余 ${days} 天到期${qtyText}${openText}，建议优先处理或补充库存。`;
},

setMessages(messages) {
  const viewMessages = messages.map(item => ({
    ...item,
    readClass: item.isRead ? "read" : "unread"
  }));
  const unreadCount = viewMessages.filter(item => !item.isRead).length;

  this.setData({
    messages: viewMessages,
    unreadCount,
    hasMessages: viewMessages.length > 0,
    hasUnread: unreadCount > 0
  });
}

```

消息中心每次根据当前药品列表动态生成。消息 ID 由状态、药品 ID 和到期日期组成，已读和已删除状态保存在本地 `messageState`。关闭 `reminderPush` 后不再生成消息。

5.10 个人中心与设置

个人中心显示药品总数、提醒数、状态数量和健康评分。评分公式为 $\max(0, 100 - \text{过期数} \times 28 - \text{临期数} \times 14)$ 。用户资料通过 `users` 集合持久化；头像先上传云存储再保存 `fileID`。设置页中的提醒开关、同步开关和时间保存在本地 `settings`。提醒时间更新函数目前只输出日志，尚未创建订阅消息或定时任务。

6. 公共服务层

`miniprogram/utils/drugService.js` 是页面与 `database` 云函数之间的适配器，集中处理调用协议和前后端字段差异。

```

function listDrugs(options = {}) {
  ...
  return callDatabase({
    action: "listDrugs",
    page: options.page || 1,
    pageSize: options.pageSize || PAGE_SIZE,
    filter: options.filter || {},
    sort: options.sort || "expiryCountdown"
  }).then(data => normalizeCloudItems(data.items || []));
}

function createDrug(drug) {
  ...
  return callDatabase({
    action: "createDrug",
    drug: toCloudDrug(drug, drug.source || "manual")
  }).then(data => normalizeCloudItem(data.drug));
}

function callDatabase(data) {
  ...
  return wx.cloud.callFunction({
    name: "database",
    data
  }).then(res => {
    const result = res.result || {};
    if (!result.success) {
      throw new Error(result.message || result.code ||
        "database call failed");
    }
    return result.data;
  });
}

function toCloudDrug(drug, source) {
  return {
    name: String(drug.name || "").trim(),
    qty: String(drug.qty || "").trim(),
    dosageForm: drug.dosageForm || "other",
    category: drug.category || "other",
    openState: toCloudOpenState(drug.openState),
    location: drug.location || "medicine_box",
    expireDateText: drug.expireDate || drug.date || dateFromDays(drug.days),
    source,
    confidence: Number(drug.confidence || 0),
    note: drug.note || "",
    rawText: drug.rawText || drug.recognizedText || "",
    attachments: drug.attachments || { images: [], audios: [] }
  };
}

```

方法	云函数 action	返回值/行为
listDrugs(options)	listDrugs	返回归一化药品数组，默认申请 pageSize=100
getDrug(id)	getDrug	返回单条归一化药品
createDrug(drug)	createDrug	将页面字段转为云端字段后创建
batchCreateDrugs(drugs)	batchCreateDrugs	批量转换并创建药品
updateDrug(id, drug)	updateDrug	更新并重新计算效期
deleteDrug(id)	deleteDrug	默认仅软删除，不删除附件
getDaysRemaining(date)	无	客户端按本地日期计算剩余整数天
getStatus(date)	无	小于 0 为 expired，0~3 为 expiring，其余 normal

6.1 字段映射

页面字段	云端字段	说明
date / expireDate	expireDateText	统一为 YYYY-MM-DD
days	daysToExpire	服务端快照返回后映射为页面 days
status	expiryStatus	服务端状态映射为页面 status
未开封/已开封/独立包装	unopened/opened/independent	客户端进行中英文枚举转换
confidence/note/rawText	ai 对象	云函数统一嵌套到 AI 元数据

7. 云函数设计

7.1 database 云函数

database 是系统的核心业务后端。入口根据 event.action 分发，使用 cloud.getWXContext() 获取 OPENID。所有成功响应使用 {success:true,data}，异常响应使用 {success:false,code,message}。

Action	输入	核心处理
listDrugs	page,pageSize,filter,sort	用户隔离、过滤、排序、分页、统计总数
getDrug	id	按 _id + _openid + 未删除查询
createDrug	drug	校验、规范化、计算效期、写入审计字段
batchCreateDrugs	drugs/items	最多 20 条，逐条规范化和写入
updateDrug	id,drug	读取归属记录、合并字段、重新计算状态
deleteDrug	id,deleteFiles	软删除，可选删除附件文件
getFileTempUrls	fileIDs	校验文件范围后生成临时 URL
deleteCloudFiles	fileIDs	校验文件范围后删除云文件
refreshExpirySnapshots	limit	最多刷新 50 条药品的剩余天数和状态
getUserProfile	无	读取用户资料，不存在时创建默认资料
updateUserProfile	profile	更新或创建昵称、头像和简介

7.2 参数规范化

- 药名必填，最长 80 字符；数量必填，最长 40 字符。
- 分类、剂型、开封状态和来源均限制在固定枚举内，非法值回退到 other、unopened 或 unknown。
- 图片最多 8 个，音频最多 4 个，仅接受 cloud:// 开头的 fileID。
- AI note 最长 200 字符，rawText 最长 1000 字符，model 最长 80 字符。
- 列表 pageSize 最大 50；关键字、位置等字段会限制长度并转义正则。
- 删除不物理移除数据库记录，而是写入 isDeleted、deletedAt 和 deletedAtTs。

7.3 aiAssistant 云函数

```
async function callTextWithFallback(messages) {
  try {
    return await callOpenAICompatible({
      url: "https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions",
      apiKey: cfg.qwenApiKey,
      model: cfg.qwenTextModel,
      messages,
      temperature: 0.3
    });
  } catch (err) {
    console.error("qwen text failed:", sanitizeError(err));
    return await callOpenAICompatible({
      url: "https://open.bigmodel.cn/api/paas/v4/chat/completions",
      apiKey: cfg.zhipuApiKey,
      model: cfg.zhipuTextModel,
      messages,
      temperature: 0.3
    });
  }
}

async function handleParseDrugText(event) {
  const text = String(event.text || "").trim();
  if (!text) throw new Error("empty text");
  const parsed = await parseDrugTextWithModel(text);
  return {
    recognizedText: text,
    items: normalizeRecognizedItems(parsed.items, text)
  };
}
```

Action	处理链路	输出
chat	整理药品摘要和消息上下文 -> 文本模型 -> 安全提示补全	reply, provider
visionRecognize	下载图片 -> Base64 Data URL -> qwen-vl-plus -> JSON 解析	items
voiceRecognize	文件临时 URL -> paraformer-v2 异步转写 -> 文本模型解析	recognizedText, items
parseDrugText	自然语言 -> 文本模型结构化解析	recognizedText, items

文本模型首先请求 DashScope OpenAI 兼容接口，失败后回退到智谱 OpenAI 兼容接口。模型默认值为 qwen-plus、qwen-vl-plus、glm-4.7 和 paraformer-v2。语音任务最多轮询 30 次，每次间隔 1 秒。

7.4 quickstartFunctions

该目录仍包含云开发模板示例，包括获取 OPENID、生成小程序码以及 sales 集合 CRUD。当前业务页面没有调用它，且小程序码路径仍指向不存在的 pages/index/index。建议在正式交付前删除或明确标记为示例，避免误部署和误用。

8. 数据库与云存储设计

8.1 drugs 集合

字段	类型	含义
_openid	string	记录所属微信用户
name	string	药品名称
nameSortKey	string	小写名称排序键
qty	string	人类可读数量
category	enum	药品分类
dosageForm	enum	剂型/包装形式
openState	enum	unopened/opened/independent
location	string	存放位置
expireDateText	string	YYYY-MM-DD 到期日期
expireDate	Date null	日期对象
expireAtTs	number	稳定到期时间戳，未知日期使用 MAX_SAFE_INTEGER
daysToExpire	number null	最近刷新时的剩余天数快照
expiryStatus	enum	expired/expiring/normal/unknown
source	enum	manual/vision/voice/migration/unknown
ai	object	confidence、note、rawText、model

字段	类型	含义
attachments	object	images 与 audios 的云文件 ID 数组
searchText	string	由名称、数量、位置和分类拼接的搜索文本
isDeleted	boolean	软删除标记
createdAt/updatedAt	Date	创建和更新时间
createdAtTs/updatedAtTs	number	创建和更新时间戳
deletedAt/deletedAtTs	Date/number	软删除审计时间

8.2 users 集合

字段	类型	说明
_openid	string	用户唯一归属标识
nickName	string	昵称，服务端最长 24 字符
avatarUrl	string	默认本地头像路径或云存储 fileID
description	string	个人简介，最长 60 字符
createdAt/createdAtTs	Date/number	创建时间
updatedAt/updatedAtTs	Date/number	更新时间

8.3 推荐索引

索引	字段	用途
owner_expiry_countdown	_openid asc, isDeleted asc, expireAtTs asc, createdAtTs desc	主列表倒计时排序
owner_status_expiry	_openid asc, isDeleted asc, expiryStatus asc, expireAtTs asc	状态筛选
owner_category_expiry	_openid asc, isDeleted asc, category asc, expireAtTs asc	分类统计
owner_updated	_openid asc, isDeleted asc, updatedAtTs desc	同步和最近变更

8.4 云存储路径

路径	内容	当前状态
ai-inputs/images/{timestamp}-{random}.{ext}	视觉识别图片	已实现，但路径未包含 OPENID
ai-inputs/audio/{timestamp}-{random}.mp3	语音识别录音	已实现，但路径未包含 OPENID
profiles/avatars/{timestamp}-{random}.{ext}	用户头像	已实现，但路径未包含 OPENID

9. 核心业务流程

9.1 手动新增

用户填写表单 -> 前端必填校验 -> drugService.toCloudDrug
-> database.createDrug -> normalizeDrugPayload
-> 计算 expireAtTs/daysToExpire/expiryStatus -> 写入 drugs -> 返回客户端模型

9.2 拍照识别

选择/拍摄图片 -> uploadFile -> aiAssistant.visionRecognize
-> 云函数下载图片 -> qwen-vl-plus -> JSON 归一化
-> 前端置信度与字段校验 -> 用户修改确认
-> database.batchCreateDrugs -> drugs 集合

9.3 语音识别

录音 -> uploadFile -> aiAssistant.voiceRecognize
-> getTempFileURL -> paraformer-v2 异步转写
-> qwen-plus / GLM 文本结构化 -> 前端确认 -> database.createDrug

9.4 提醒生成

读取 drugs -> 过滤 expired/expiring -> 按紧急度排序

-> 首页显示最多 3 条 / 消息中心生成完整消息

-> messageState 叠加本地已读与删除状态

9.5 数据刷新

用户在个人中心或设置页触发 refreshExpirySnapshots。云函数按 updatedAtTs 升序读取最多 50 条未删除记录，重新计算 daysToExpire 和 expiryStatus，并更新 updatedAt。当前没有自动定时触发器，因此长期不编辑的记录依赖用户主动刷新或页面端重新计算。

10. 效期计算与状态规则

```
function buildExpirySnapshot(dateText, today) {
  if (!dateText) {
    return {
      expireDate: null,
      expireAtTs: Number.MAX_SAFE_INTEGER,
      daysToExpire: null,
      expiryStatus: "unknown"
    };
  }

  const expireDate = parseLocalDate(dateText);
  const daysToExpire = Math.floor((expireDate.getTime() - today.getTime()) / 86400000);
  return {
    expireDate,
    expireAtTs: expireDate.getTime(),
    daysToExpire,
    expiryStatus: daysToExpire < 0 ? "expired" : daysToExpire <= 3 ? "expiring" : "normal"
  };
}
```

条件	状态	展示
未提供日期	unknown	日期未知
daysToExpire < 0	expired	已过期 N 天
0 <= daysToExpire <= 3	expiring	临期，剩余 N 天
daysToExpire > 3	normal	正常，剩余 N 天

服务端将日期标准化为本地日历日的开始时间，并使用 `floor((expireDate - today) / 86400000)` 计算剩余天数。未知日期的 `expireAtTs` 设置为 `Number.MAX_SAFE_INTEGER`，使其在升序列表中排到最后

11. 本地存储与状态管理

键	内容	生命周期/用途
<code>isLogin</code>	字符串 <code>true</code>	本地登录标记
<code>loginTime</code>	毫秒时间戳	7 天登录有效期判断
<code>userInfo</code>	昵称、头像、简介	登录后缓存；云端 <code>users</code> 才是资料来源
<code>settings</code>	<code>reminderPush</code> 、 <code>dataSync</code> 、 <code>remindTime</code>	仅本机偏好
<code>messageState</code>	<code>readMap</code> 、 <code>deletedMap</code>	提醒已读和隐藏状态
<code>aiChatMessages</code>	最近 30 条消息	AI 会话缓存
<code>drugFilter</code>	一次性筛选条件	跨 Tab 导航参数，读取后删除
<code>focusDrugId</code>	药品 ID	聚焦目标记录，读取后删除

项目没有使用 `Redux`、`MobX` 等全局状态库。页面级 `data`、云数据库和 `wx Storage` 共同组成状态来源。跨页面短期状态通过一次性存储键传递。

12. AI 能力设计

12.1 输出约束

AI 识别结果会被 `normalizeRecognizedItems` 限制为最多 8 条，并统一输出 `name`、`qty`、`category`、`location`、`openState`、`date`、`confidence`、`note`。分类不在枚举内时回退为 `other`；日期必须匹配 `YYYY-MM-DD`；置信度被限制在 0 到 1。

12.2 模型回退

文本请求优先使用 `qwen-plus`。当请求异常时记录脱敏错误并切换到智谱 GLM。视觉识别仅使用 `qwen-vl-plus`，没有第二视觉提供商。语音识别依赖 DashScope 异步任务，提交成功后轮询任务状态。

12.3 安全提示

AI 对话返回前执行 `ensureSafetyNote`。如果回答中未检测到医生、药师或不替代医嘱等关键词，会自动追加“仅用于药箱管理参考”的提示。该机制属于文本兜底，不应替代更严格的内容安全、医学知识审核和风险分类。

12.4 配置管理

`config.example.js` 提供占位配置，实际 `index.js` 直接 `require('./config')`。当前 `config.json` 仅配置云函数运行参数，模型密钥应位于未提交的 `config.js` 或云函数环境变量/密钥管理服务中。任何真实 API Key 都不应写入版本库、文档或前端代码。

13. 异常处理与降级

场景	当前处理	用户体验
云数据库不可用	捕获 Promise 异常，记录 <code>console.error</code>	显示加载失败或检查云函数提示
图片上传失败	停止 AI 请求并创建空白候选	用户仍可手动填写
视觉识别失败	创建空白候选并打开编辑面板	保留手工录入路径
语音识别失败	保留当前表单	提示可手动填写或输入文本解析
AI 对话失败	追加固定降级回复	引导查看药品列表
删除失败	不修改云端记录，Toast 提示	列表数据保持原状
资料加载失败	使用默认资料/错误状态	其他功能仍可访问

14. 未来优化方向

14.1 安全与正确性

- 迁移 AI 密钥到云端环境变量/密钥服务并轮换现有密钥。

- 所有上传路径加入 OPENID，收紧文件所有权验证。
- 实现药品列表分页或游标加载，解决 50 条上限。
- 将批量新增改为事务或记录逐项结果，支持失败重试。
- 统一效期计算到服务端，并明确 Asia/Shanghai 时区。

14.2 提醒与同步

- 接入微信订阅消息并获得用户明确授权。
- 通过云函数定时触发器每日刷新效期快照并生成提醒。
- 将消息已读/删除状态迁移到云端 messages 或 user_message_state 集合。
- 实现设置项云端同步，同时保留本地缓存加速。

14.3 工程化

- 抽取 enums/constants 模块，统一分类、剂型、开封状态和阈值。
- 为云函数核心纯函数增加单元测试，为页面关键流程增加自动化测试。
- 加入 ESLint、格式化、敏感信息扫描和 CI。
- 重写 README，补充部署脚本、环境矩阵和变更记录。
- 删除 quickstartFunctions 或迁移到 examples 目录。

14.4 产品能力

- 支持扫码录入、OCR 原文预览、药品说明书关联和附件管理。
- 支持家庭成员共享药箱、角色权限和操作审计。
- 支持开封后有效期、批次、库存单位和用量记录。
- 增加统计看板、过期损耗分析和补货建议。
- 对 AI 结果建立人工反馈闭环和可观测指标。

当前项目已经形成可运行的微信云开发药品效期管理闭环，核心 CRUD、用户隔离、多模态录入和 AI 对话均有代码实现。正式上线前应优先处理密钥管理、文件归属、分页、真实提醒和自动化测试问题。